前置知识

存放一些Python知识或者一些其他工程开发的知识?

dataclass

dataclass 是 Python 3.7+ 标准库 dataclasses 提供的一个装饰器，用来简化以存储数据为主的类的编写。它会根据类中的类型注解自动生成一些常用方法
基础的用法是

```
from dataclasses import dataclass

@dataclass
class Point:
    x: int
    y: int
    z: int = 0

p = Point(1, 2)
print(p)        # Point(x=1, y=2, z=0)
print(p.x)      # 1
print(p == Point(1, 2))  # True
```

可以像这样可以使用参数指定规则

```
@dataclass(order=True)
class Item:
    price: float
    name: str
```

field()

dataclasses提供的一个工具函数，用来给某个字段做额外配置，也就是给某个字段设置规则
比如

```
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str


```

```
    # field(default_factory=list) 的意思是：
    # 每次创建 Student 对象时，都调用 list() 生成一个新的空列表作为默认值。
    # 这样每个实例的 scores 都是独立的，不会共享同一个列表。
    scores: list[int] = field(default_factory=list)

    # field(init=False) 表示这个字段不会出现在 __init__ 参数里，
    # 也就是创建对象时不能直接传入 id，比如 Student("Alice", id=1) 会报错。
    # default=0 表示这个字段的默认值是 0。
    # 通常用于内部自动生成或后续赋值。
    id: int = field(init=False, default=0)
```

args命令解析器

argparse是Python标准库中用来解析命令行参数的模块。比如我们使用的git命令就是这类argparse工具在解析
它在Python有以下的函数操作

```
parser = argparse.ArgumentParser(...)      #构造命令解析器
parser.add_argument("--version", ...)      # 给解析器登记一条规则，这里只要解析器从命令行中识别到--version就会执行后续将version置为true的操作
subparsers = parser.add_subparsers(...)    #给解析器登记“子命令”能力
args = parser.parse_args()                 #启动解析器并将结果获取到args中
```

因为我们涉及到uv，所以这里讲清楚所以情况

```
uv run kama --version # 命令通过，uv识别到kama，解析器的规则识别到--version
uv run kama ping #命令通过 ，uv识别到kama，解析器识别到子命令ping，并且ping登记过
uv run gama --version #命令不通过，uv没有识别到登记的prog中的"kama"
uv run kama ping --version #命令不通过，在识别到ping之后进入了ping的子解析器，而ping的子解析器中并没有--version的子命令，这里不会把--version识别成规则，而是ping的子命令
uv run kama --version wait,what? #命令不通过，因为没有wait,what?的子命令，我们没注册过
uv run kama --version ping #命令通过，执行返回版本号，规则与子命令都成功识别
```

应该是把全部情况讨论出来，各位也能理解到底是怎么实现的了